Software Engineering Practice Quiz 1

Name: Wang Hao (汪浩)
NUIST ID: 202383890022 SETU ID: 20109992

1 What is Analysis and Design?

Analysis investigates the problem, needs and domain: what must the system do, and which concepts matter? Design defines a solution that satisfies those needs, including responsibilities, interfaces and collaborations. For example, analysis identifies Sale and refund eligibility; design assigns refund validation to PosController and subtotal calculation to ReturnLineItem. Analysis and design evolve together during iterations.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 1.

2 What is the UML?

The Unified Modeling Language is a standard visual language for specifying, visualizing and documenting software and related systems. It provides notation such as class, sequence and package diagrams. UML is a modeling language, not a development process, and does not itself explain how to assign good responsibilities.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 1.

3 What is Iterative and Evolutionary Development?

Iterative development organizes work into repeated, timeboxed cycles that each produce an integrated, tested increment. Evolutionary development lets requirements and design grow through learning and feedback rather than assuming they are complete at the beginning. A POS iteration can deliver cash sales first and a later iteration add returns while refining the shared model.

Reading: Larman, Applying UML and Patterns, 3rd ed., Chs. 2 and 5.

4 How Long Should an Iteration Be? Why?

Larman recommends short, fixed-length iterations, commonly two to six weeks in the Unified Process, with shorter periods often preferred. The appropriate length depends on risk and context. Short iterations expose problems, produce feedback and limit the cost of wrong assumptions. Timeboxing normally changes scope rather than extending the deadline. This assignment explicitly permits three-day or one-week sprints.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 2; assignment sprint guidance.

5 Why are feedback and adaptation key ingredients for success in complex, changing systems?

Requirements, technical constraints and stakeholder understanding are incomplete at the start. Working increments and tests reveal information that documents alone cannot establish. Feedback allows the team to correct priorities, requirements and design early, reducing the cost of developing the wrong solution. For example, a return demo may reveal that original prices must be stored instead of reading current catalog prices.

Reading: Larman, Applying UML and Patterns, 3rd ed., Chs. 2 and 5.

6 What is Agile Modeling?

Agile Modeling uses lightweight, collaborative modeling practices to understand a problem or communicate a design. Teams create only models that have a useful purpose, use several complementary views and refine them as needed. A whiteboard sequence sketch may be sufficient to resolve a responsibility question. Agility does not mean eliminating analysis or keeping every diagram permanently complete.

Reading: Larman, Applying UML and Patterns, 3rd ed., Chs. 5 and 14.

7 What is a Domain Model? Why Create a Domain Model?

A domain model is a visual representation of significant real-world conceptual classes, their attributes and associations. It describes the problem domain rather than software methods or database design. It builds a shared vocabulary, exposes missing concepts and informs later responsibility assignment. A POS model may include Sale, SalesLineItem, ProductDescription, Return and CashPayment.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 9.

8 What is the Logical Architecture? And Layers?

Logical architecture is the large-scale organization of software classes into packages, subsystems and layers. It describes logical responsibilities and dependencies rather than where executables are deployed. A layer groups related responsibilities at a level of abstraction, such as presentation, application coordination, domain logic or persistence. Logical layers and physical deployment tiers are different concepts.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 13.

9 What are the essential ideas of using layers in software architecture design? What are benefits of using layers?

Assign each layer a coherent concern and restrict dependencies so higher-level services can use lower-level services without lower layers depending on the UI. Keep domain rules separate from presentation and infrastructure details. Layering reduces the effect of changes, supports reuse, improves testability and makes responsibilities easier to understand. Dependency rules can be strict or relaxed, but should remain explicit.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 13.

10 What is the Model-View Separation Principle?

Keep domain model objects independent of user-interface objects and UI presentation logic. A view may obtain model state and invoke application operations, but a Sale should not read console input or draw a screen. Separate rules can then support several interfaces and automated tests. If views need notifications, use a controlled notification mechanism rather than introducing direct model-to-widget dependencies.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 13.

11 What are Static and Dynamic Modeling in OO design?

Static modeling describes software structure, such as classes, attributes, operations, associations and dependencies. Dynamic modeling describes behavior and object collaboration over time, using sequence or communication diagrams. The views inform one another: a message in an interaction suggests an operation and required visibility in the class model. Both should be refined iteratively.

Reading: Larman, Applying UML and Patterns, 3rd ed., Chs. 15 and 16.

12 What are the Strengths and Weaknesses of Sequence vs. Communication Diagrams in OO design?

Sequence diagrams make message order, lifetimes and conditional or repeated interaction easier to follow along a time axis. They can become wide and make the overall network of object links less obvious. Communication diagrams emphasize object links and use numbered messages, making structural collaboration compact. Complex ordering and nested control flow are harder to read there. Both represent interactions and can describe the same collaboration.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 15.

13 What are advantages of Test-Driven Development?

In TDD, write a failing test for a small behavior, implement enough code to pass, then refactor while tests remain green. Benefits include fast regression feedback, precise examples of required behavior and pressure toward small, testable interfaces. Tests also support safer refactoring. These benefits depend on meaningful tests; TDD does not replace acceptance testing, exploratory testing or architectural thinking.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 21.

14 Please give the definition of the Creator pattern.

Assign class B responsibility for creating an instance of class A when B contains or aggregates A, records A, closely uses A, or has the initializing data needed for A. Prefer the containing or aggregating object when applicable. This tends to use existing knowledge and limit coupling. In the POS design, Sale creates SalesLineItem because it contains its lines.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 17.

15 What is the Factory pattern for OO design? When to use Factory pattern?

A factory centralizes object creation so clients can request suitable objects without knowing concrete classes or complex construction details. Use it when creation logic is complex, varies by configuration, or would otherwise create excessive coupling. A factory could select a payment-service adapter. A simple factory, Factory Method and Abstract Factory are related creation approaches with different structures; a trivial constructor does not require a factory.

Reading: Larman, Applying UML and Patterns, 3rd ed., Ch. 26.